

```

64 var x = o.left;
65 var y = o.top;
66
67 var ax = settings.accX;
68 var ay = settings.accY;
69 var th = t.height();
70 var wh = w.height();
71 var tw = t.width();
72 var ww = w.width();
73
74 if (y + th + ay >= b &&
75     y <= b + wh + ay &&
76     x + tw + ax >= a &&
77     x <= a + ww + ax) {
78     //trigger the custom event
79     if (!t.appeared) t.trigger('appear', settings.data);
80
81     } else {
82     //it scrolled out of view
83     t.appeared = false;
84     }
85
86 };
87
88 //create a modified fn with some additional logic
89 var modifiedFn = function() {
90
91     //mark the element as visible
92     t.appeared = true;
93
94     //is this supposed to happen only once?
95     if (settings.one) {
96
97         //remove the check
98         w.unbind('scroll', check);
99         var i = $.inArray(check, $.fn.appear.checks);
100         if (i >= 0) $.fn.appear.checks.splice(i, 1);
101     }
102
103     //trigger the original fn
104     fn.apply(this, arguments);
105
106 };
107
108 //bind the modified fn to the element
109 $.fn.appear.one('appear', settings.data, modifiedFn);
110
111 //bind the modified fn to the element
112 $.fn.appear.one('appear', settings.data, modifiedFn);

```



MATERIA:

Programación Orientada a Objetos

Unidad 2 Principios de la Programación Orientada a Objetos.

Docente:

Grupo:

Unidad #2 - Principios de la Programación Orientada a Objetos.

2.7. Encapsulamiento.

2.7.1. Uso de clases internas y anidadas

2.7.2 Genericos

2.7.3. Paquetes.

- 2.7.3.1 **Organización estructurada de proyectos Java modernos siguiendo convenciones de paquetes

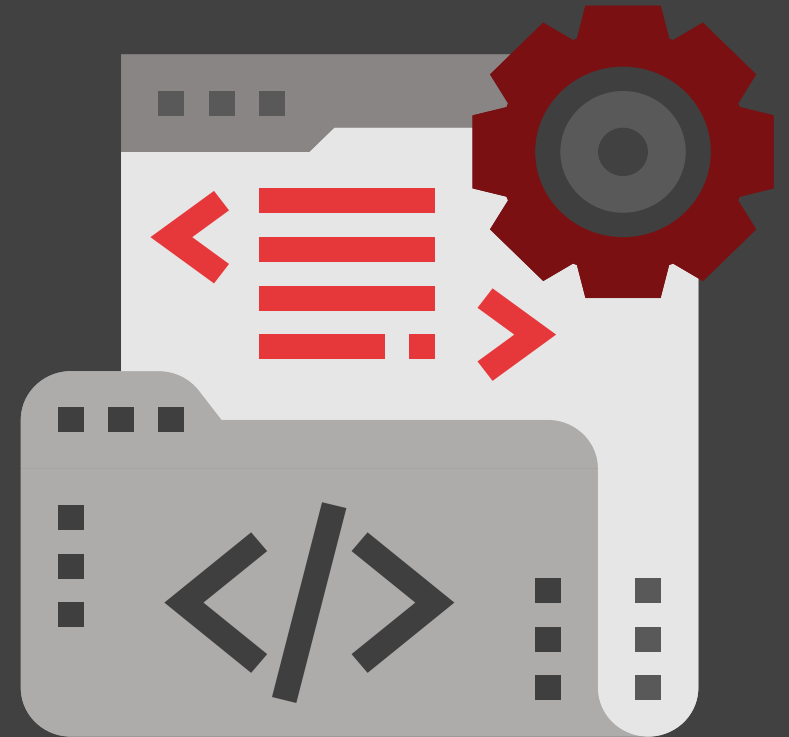
2.7. Encapsulamiento.

El **encapsulamiento** es uno de los **cuatro pilares fundamentales** de la **Programación Orientada a Objetos (POO)** junto con la herencia, el polimorfismo y la abstracción.

Su idea central es **ocultar los detalles internos de una clase** y **exponer únicamente lo necesario** para que los demás objetos interactúen con ella de manera controlada.

En programación, implica **proteger los atributos y métodos internos de una clase** para evitar accesos indebidos o modificaciones no controladas.

- Pilar fundamental de la POO.
- **Ocultar los detalles internos** de una clase.
- Los atributos se declaran **privados**.
- Se accede mediante **métodos públicos (getters y setters)**.



2.7. Encapsulamiento.

✗ Sin Encapsular

```
class Cuenta {  
    public String titular;  
    public double saldo;  
}
```

¡Cualquiera puede modificar!

✓ Encapsulado

```
class Cuenta {  
    private String titular;  
    private double saldo;  
  
    public double getSaldo() {  
        return saldo;  
    }  
}
```

Acceso controlado 🔒

- **Sin Encapsular:** Los atributos `titular` y `saldo` son públicos. Cualquiera puede acceder y modificarlos directamente, lo que es inseguro.
- **Encapsulado:** Los atributos son privados (`private`) y solo se accede a ellos mediante métodos (por ejemplo, `getSaldo()`). Esto da **acceso controlado**, protege los datos y evita cambios indebidos.

Encapsular = proteger datos + controlar el acceso.

2.7. Encapsulamiento.

```
class CuentaBancaria { 2 usages new *
    private String titular; 3 usages
    private double saldo; 5 usages

    // Constructor
    public CuentaBancaria(String titular, double saldoInicial) {
        this.titular = titular;
        if (saldoInicial >= 0) {
            this.saldo = saldoInicial;
        }
    }

    // Getter para titular
    public String getTitular() { 1 usage new *
        return titular;
    }

    // Setter para titular
    public void setTitular(String titular) { no usages new *
        if (titular != null && !titular.isEmpty()) {
            this.titular = titular;
        }
    }
}
```

```
// Getter para saldo
public double getSaldo() { 1 usage new *
    return saldo;
}

// Método controlado para depositar
public void depositar(double monto) { 1 usage new *
    if (monto > 0) {
        saldo += monto;
    }
}

// Método controlado para retirar
public void retirar(double monto) { 1 usage new *
    if (monto > 0 && monto <= saldo) {
        saldo -= monto;
    } else {
        System.out.println("Fondos insuficientes");
    }
}
}
```

```
public class Main { @franciscomorales25 *
    public static void main(String[] args) { @franciscomorales25 *
        CuentaBancaria cuenta = new CuentaBancaria( titular: "Ana López", saldoInicial: 1000);

        cuenta.depositar( monto: 500); // saldo = 1500
        cuenta.retirar( monto: 200); // saldo = 1300

        System.out.println("Titular: " + cuenta.getTitular());
        System.out.println("Saldo actual: " + cuenta.getSaldo());
    }
}
```

Los atributos **titular** y **saldo** son **privados**.

Se accede a ellos mediante **métodos públicos controlados**.

Se agregan validaciones (ej. no saldo negativo, no retirar más de lo disponible).

2.7. Encapsulamiento. Modificadores de acceso

PRIVATE

Solo dentro de la clase

```
private String password;
```

★ Más restrictivo

DEFAULT

Mismo paquete

```
String nombre; // sin modificador
```

📦 Package-private

PROTECTED

Paquete + Subclases

```
protected int edad;
```

👤 Herencia

PUBLIC

Desde cualquier lugar

```
public void mostrar() {...}
```

🌐 Sin restricciones

Private protege totalmente los datos.

Default da acceso solo en el paquete.

Protected abre un poco más, incluyendo herencia.

Public da acceso sin limitaciones.

2.7.1. Clases Internas y Anidadas

Una **clase interna** es una clase definida dentro de otra clase.

Sirven para **agrupar lógicamente** clases que solo tienen sentido juntas.

Clases anidadas:

- **Estáticas (static nested class):** funcionan como clases independientes pero dentro de otra.
- **Internas (non-static inner class):** dependen de la instancia de la clase externa.

Ventajas

- ✓ Mejor **organización del código**.
- ✓ Favorece el **encapsulamiento** (acceso directo a miembros privados de la clase externa).
- ✓ Útiles para implementar **estructuras de datos** o clases de apoyo.

2.7.1. Clases Internas y Anidadas

Inner Class

Asociada a instancia

```
class Universidad {  
    private String nombre;  
  
    class Facultad {  
        void mostrar() {  
            // Acceso a nombre  
        }  
    }  
}
```

Static Nested

Independiente

```
class Universidad {  
    static class Config {  
        static String version;  
    }  
}  
  
// Uso:  
Universidad.Config.version
```

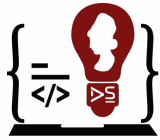
Inner Class (Clase Interna)

- **Asociada a la instancia** de la clase externa.
- Puede **acceder directamente** a los atributos y métodos privados de la clase externa.
- Necesita que exista un **objeto de la clase externa** para ser usada.

Static Nested Class (Clase Anidada Estática)

- **Independiente** de la instancia de la clase externa.
- Solo puede acceder a los **miembros estáticos** de la clase externa.
- Se accede usando la **notación ClaseExterna.ClaseAnidada**.

Ejemplo: Clase Interna (Inner Class)



Ingeniería en
DESARROLLO
DE SOFTWARE

```
class Universidad { 3 usages new *
    private String nombre; 2 usages

    // Constructor
    public Universidad(String nombre) { 1 usage new *
        this.nombre = nombre;
    }

    // Clase interna
    class Facultad { 2 usages new *
        private String facultad; 2 usages

        public Facultad(String facultad) { 1 usage new *
            this.facultad = facultad;
        }

        public void mostrarInfo() { 1 usage new *
            // La clase interna puede acceder a "nombre" de Univ
            System.out.println("Universidad: " + nombre);
            System.out.println("Facultad: " + facultad);
        }
    }
}
```

```
public class Main { @ franciscomorales25 *
    public static void main(String[] args) { @ franciscomorales25 *
        // Crear objeto de clase externa
        Universidad u = new Universidad( nombre: "UES");
        // Crear objeto de clase interna a través de la externa
        Universidad.Facultad f = u.new Facultad("Ingeniería");

        f.mostrarInfo();
    }
}
```

Facultad es una clase interna de Universidad.

Puede acceder a los atributos privados de Universidad (nombre).

Para instanciarla: Universidad.Facultad f = u.new Facultad("Ingeniería")

Ejemplo práctico:

- Una clase Universidad puede tener la clase interna Facultad, porque solo tiene sentido dentro de esa estructura.
- Así, evitas crear Facultad como clase independiente en todo el proyecto.

2.7.2 Genericos

- Permiten **definir clases, interfaces y métodos** con **tipos de datos como parámetros**.
- Evitan **duplicar código** para distintos tipos de datos.
- Se representan con **<T>**, donde **T** es un **parámetro de tipo**.

Ventajas de los Genéricos

Seguridad de tipos (type safety): evita errores en tiempo de ejecución.

Reutilización de código: una clase sirve para múltiples tipos.

Legibilidad: más claro y ordenado que usar **Object**.

Menos casting: no se necesita convertir objetos manualmente.

Ejemplo de Clase Genérica ➡

```
// Clase genérica con un parámetro de tipo <T>
class Caja<T> { 4 usages new *
    private T contenido; 2 usages

    public void setContenido(T contenido) { 2 usages new *
        this.contenido = contenido;
    }

    public T getContenido() { 2 usages new *
        return contenido;
    }
}

public class Main { 2 franciscomorales25 *
    public static void main(String[] args) { 2 franciscomorales25 *
        Caja<String> caja1 = new Caja<>();
        caja1.setContenido("Libro");

        Caja<Integer> caja2 = new Caja<>();
        caja2.setContenido(100);

        System.out.println(caja1.getContenido()); // Libro
        System.out.println(caja2.getContenido()); // 100
    }
}
```

2.7.2 Genericos

```
class Caja {  
    private Object contenido;  
  
    public void guardar(Object obj) {  
        contenido = obj;  
    }  
  
    public Object obtener() {  
        return contenido;  
    }  
}
```

```
Caja caja = new Caja();  
caja.guardar("Hola");  
String texto = (String) caja.obtener(); // ⚠ Cast manual  
caja.guardar(123);  
String error = (String) caja.obtener(); // ✖ Error en ejecución
```

Problema Sin Genéricos

Sin genéricos: inseguro, requiere cast manual, errores en ejecución.

Con genéricos: seguro, sin cast, errores detectados en compilación.

```
class Caja<T> {  
    private T contenido;  
  
    public void guardar(T obj) {  
        contenido = obj;  
    }  
  
    public T obtener() {  
        return contenido;  
    }  
}
```

```
Caja<String> cajaTexto = new Caja<>();  
cajaTexto.guardar("Hola");  
String texto = cajaTexto.obtener(); // ✅ Sin cast  
  
Caja<Integer> cajaNumero = new Caja<>();  
cajaNumero.guardar(123);  
// cajaNumero.guardar("error"); // ❌ Error en compilación
```

Solución: Genéricos

2.7.3. Paquetes.

Un **paquete (package)** en Java es un **mecanismo de organización** que agrupa clases, interfaces y subpaquetes.

Sirve para:

- ✓ Mantener el código organizado.
- ✓ Evitar conflictos de nombres.
- ✓ Reutilizar componentes.

```
package nombrePaquete;  
  
public class MiClase {  
    // código  
}
```

El paquete debe coincidir con la **estructura de carpetas** del proyecto.

Importar Paquetes

```
import java.util.ArrayList;  
import java.util.Scanner;
```

- `import` permite usar clases de otros paquetes.
- Se pueden importar clases específicas o todo el paquete (*).

Ventajas de los Paquetes

- ✓ Organización modular del código.
- ✓ Evitan conflictos de nombres.
- ✓ Facilitan el mantenimiento.
- ✓ Permiten reutilizar clases y librerías.

Tipo de Paquetes

Paquetes Integrados (Built-in Packages)

- Incluidos en la **librería estándar de Java (JDK)**.
- Listos para usar en proyectos.
- Ejemplos:
 - `java.util` → colecciones (ArrayList, HashMap).
 - `java.io` → manejo de archivos.
 - `java.sql` → bases de datos.
 - `java.net` → redes.

Paquetes Definidos por el Usuario (User-defined Packages)

- Creados por el programador.
- Útiles en proyectos grandes para separar **módulos**.

```
java

package com.escuela;

public class Estudiante {
    private String nombre;
    public Estudiante(String n) { nombre = n; }
    public void mostrar() { System.out.println(nombre); }
}
```

```
import com.escuela.Estudiante;

public class Main {
    public static void main(String[] args) {
        Estudiante e = new Estudiante("Ana");
        e.mostrar();
    }
}
```

Ejemplo paquetes

```
package com.escuela;

public class Estudiante { 3 usages new *
    private String nombre; 2 usages
    public Estudiante(String n) { nombre = n; } 1 usage
    public void mostrar() { System.out.println(nombre);
}

// Archivo: Main.java
import com.escuela.Estudiante;

public class Main { 2 franciscomorales25 *
    public static void main(String[] args) { 2 franciscom
        Estudiante e = new Estudiante( n: "Ana");
        e.mostrar();
    }
}
```

Indica que la clase `Estudiante` pertenece al paquete `com.escuela`.
Eso implica que el archivo debe estar guardado en la carpeta:

 `com` →  `escuela` → `Estudiante.java`.

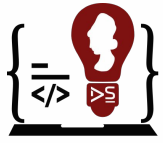
Organización estructurada de proyectos Java modernos siguiendo convenciones de paquetes

En proyectos Java modernos, **los paquetes no solo agrupan clases**, sino que también representan la **estructura lógica y modular de la aplicación**.

Una buena organización de paquetes:

- ✓ Facilita el **mantenimiento y escalabilidad**.
- ✓ Promueve el **trabajo en equipo** al dividir responsabilidades.
- ✓ Permite seguir **estándares internacionales** usados por empresas y frameworks.

Convención actual: usar el **dominio de la organización en orden inverso**, seguido del nombre del proyecto y sus módulos.



Organización estructurada de proyectos Java moderno

□ Estructura de Proyecto Java (Maven/Gradle)

proyecto/

```
├── src/
│   ├── main/
│   │   ├── java/
│   │   │   └── sv/edu/ues/
│   │   │       ├── modelo/      (Entidades de dominio)
│   │   │       ├── servicio/    (Lógica de negocio)
│   │   │       ├── repositorio/ (Acceso a datos)
│   │   │       ├── controlador/ (Manejo de peticiones)
│   │   │       └── util/         (Clases auxiliares)
│   │   └── resources/ (Config. y plantillas)
│   └── test/             (Pruebas)
└── pom.xml              (Configuración Maven)
```

- 📁 **modelo/** → Entidades y objetos de dominio (POJOs).
- 📁 **repositorio/** → Acceso y persistencia en base de datos.
- 📁 **servicio/** → Lógica de negocio y reglas del sistema.
- 📁 **controlador/** → Manejo de peticiones (API REST, GUI).
- 📁 **util/** → Clases auxiliares, validaciones, helpers.

Organización estructurada de proyectos Java moderno

Ventajas de esta organización

- ✓ Código ordenado por **capas**.
- ✓ Facilita el **trabajo en equipo** → cada grupo trabaja en su paquete.
- ✓ Compatible con **IDE modernos** (Eclipse, IntelliJ, NetBeans).
- ✓ Es el **estándar en proyectos empresariales** (ej. aplicaciones web en Spring Boot).